

Getting Started

Introduction

This chapter describes how to create a project and import the jar package

Install Android Studio

To install Android Studio, visit <https://developer.android.com/studio/> and click DOWNLOAD ANDROID STUDIO. The project uses the following configuration:

1. Minimum SDK version - API 19: Android 5.1 (Lollipop)
2. Target SDK version - API 30 : Android 11 (Red Velvet Cake)
3. Gradle version - 6.5 or the latest version that matches your Android Studio version.
4. Android Gradle plugin version 4.1.3 or the latest version that matches your Android Studio version, defined in the top-level build.gradle file.
5. Java version: 1.8 (Java 8)
6. Android Studio 2.3 and higher.

Note: The latest SDK tools included with Android Studio are acceptable. To download using the SDK Manager

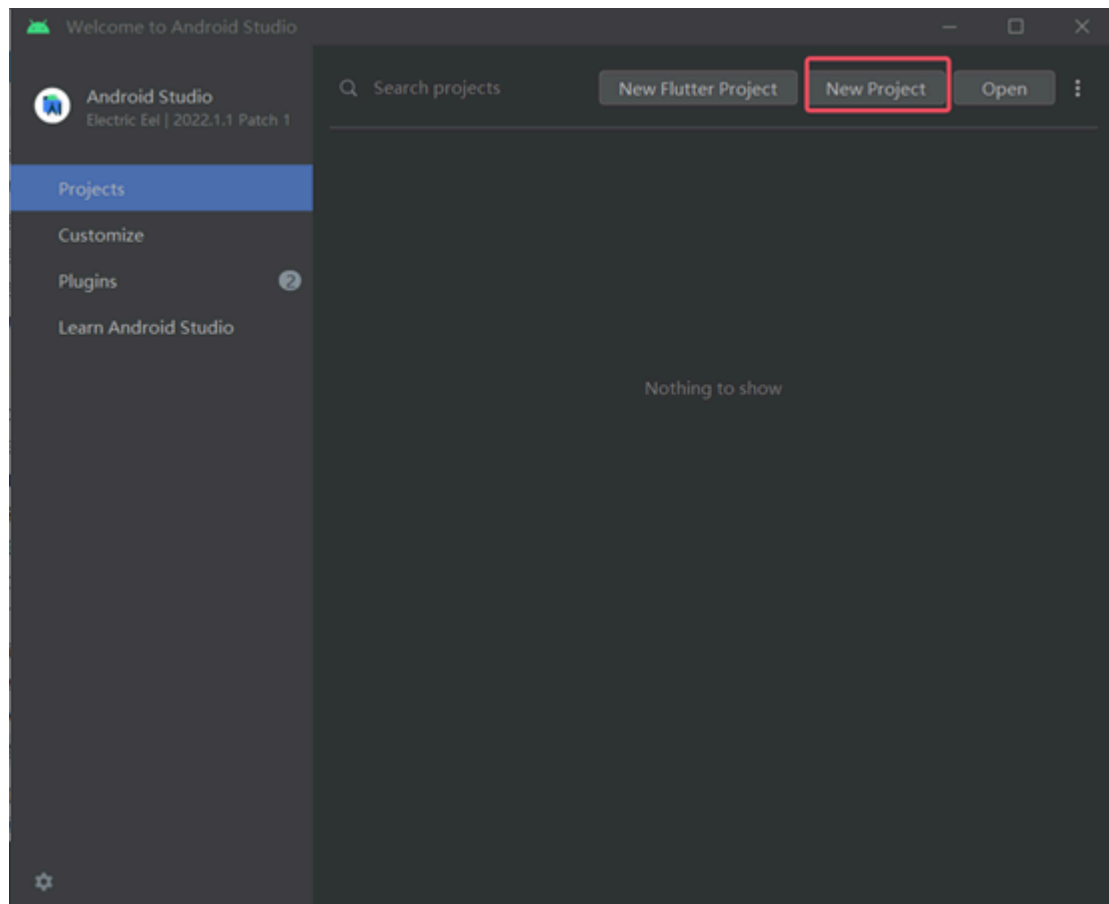
Required Android SDK packages go to menu Tools > Android > SDK Manager

Create a project

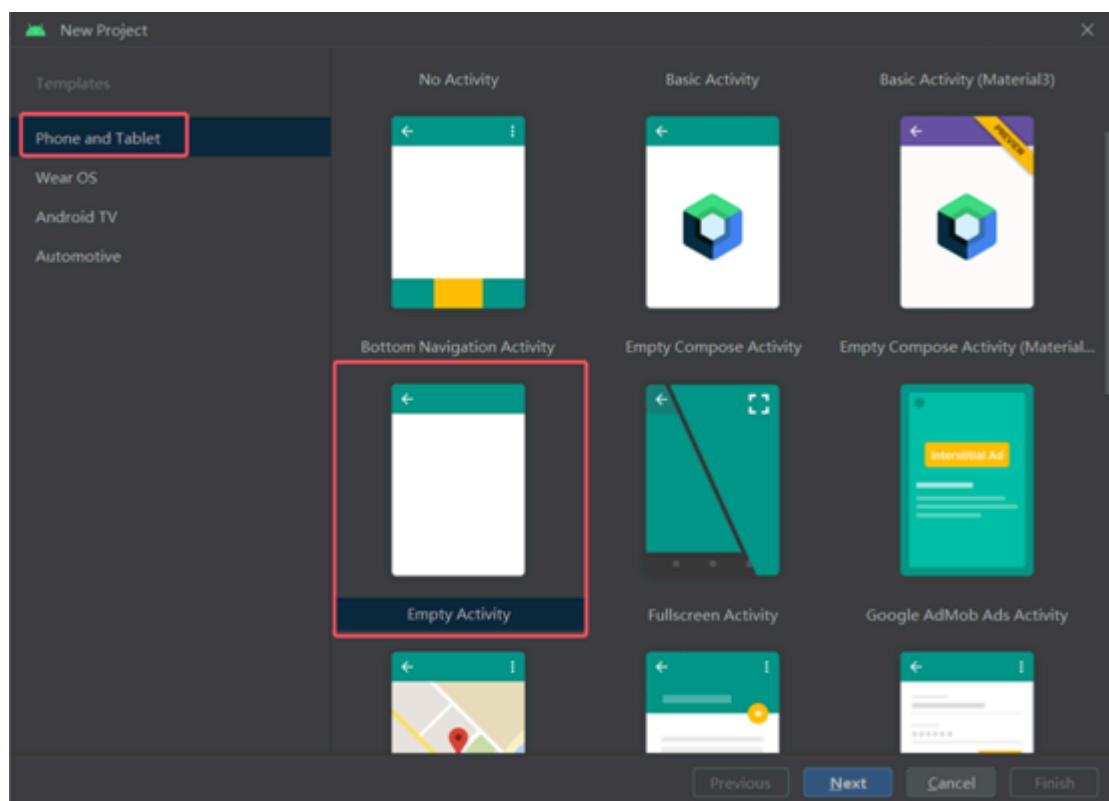
When you first install and open Android Studio, you may first start downloading the required Gradle/SDK packages. Define proxy information

Android Studio and gradl properties, then Android Studio starts building the project .

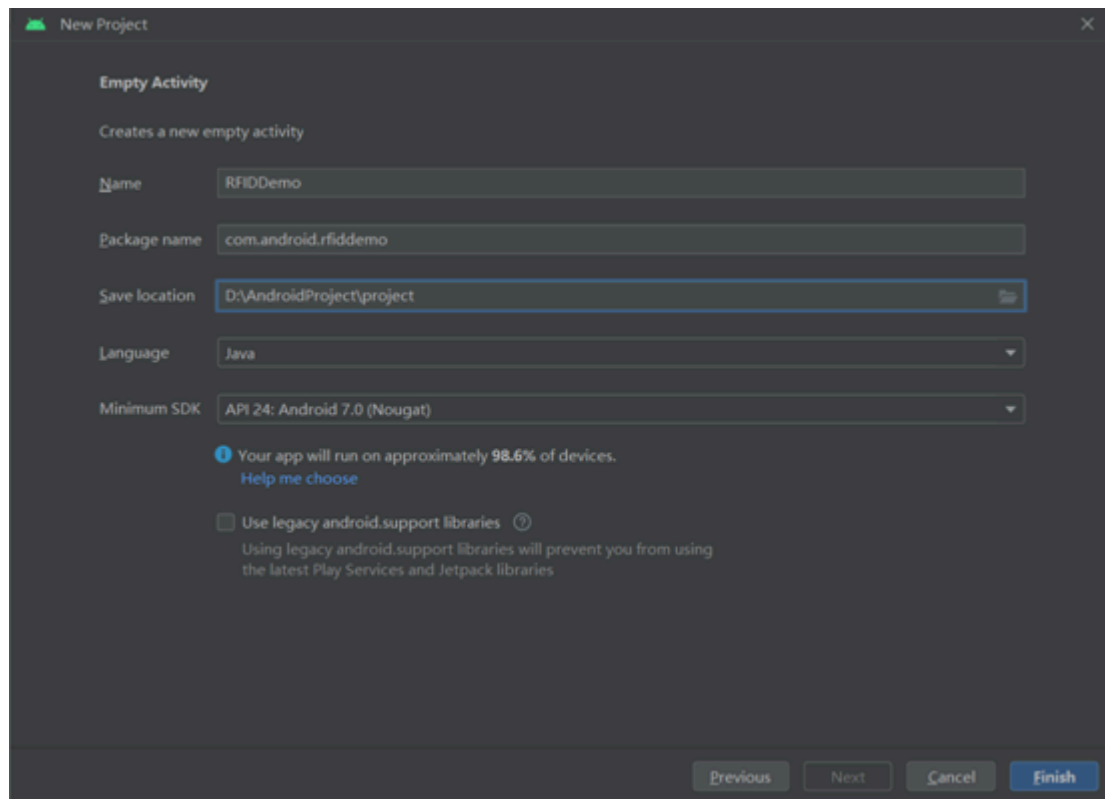
1. Click New-Project to enter the project creation interface.



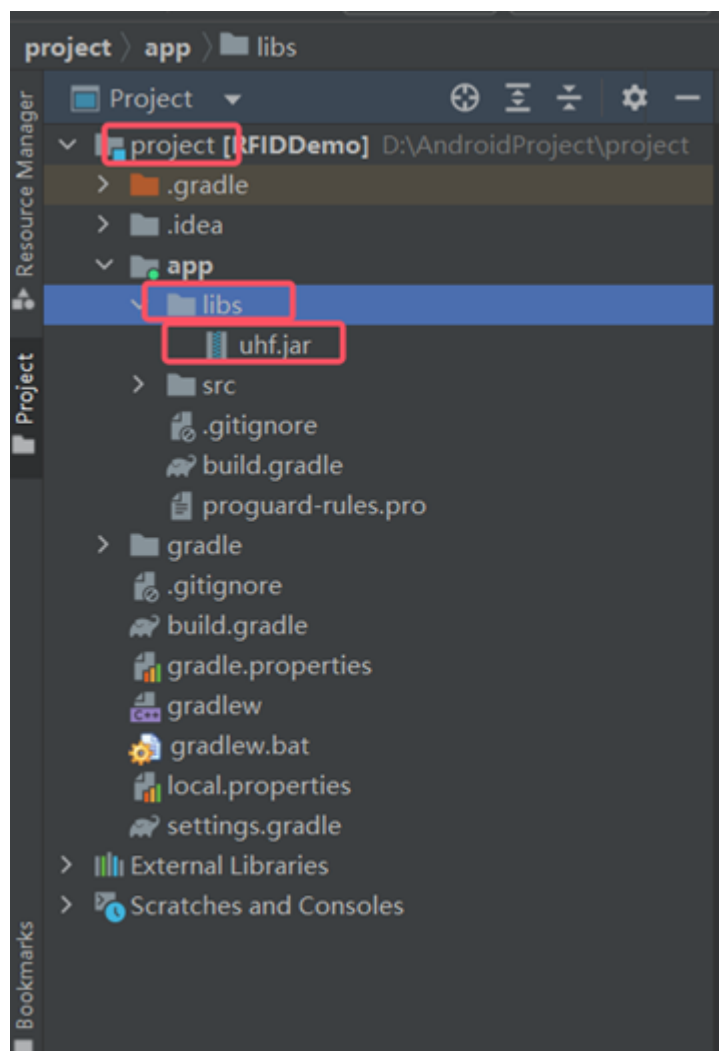
2. Select Phone and Tablet, select Empty Activity on the right, and click Next



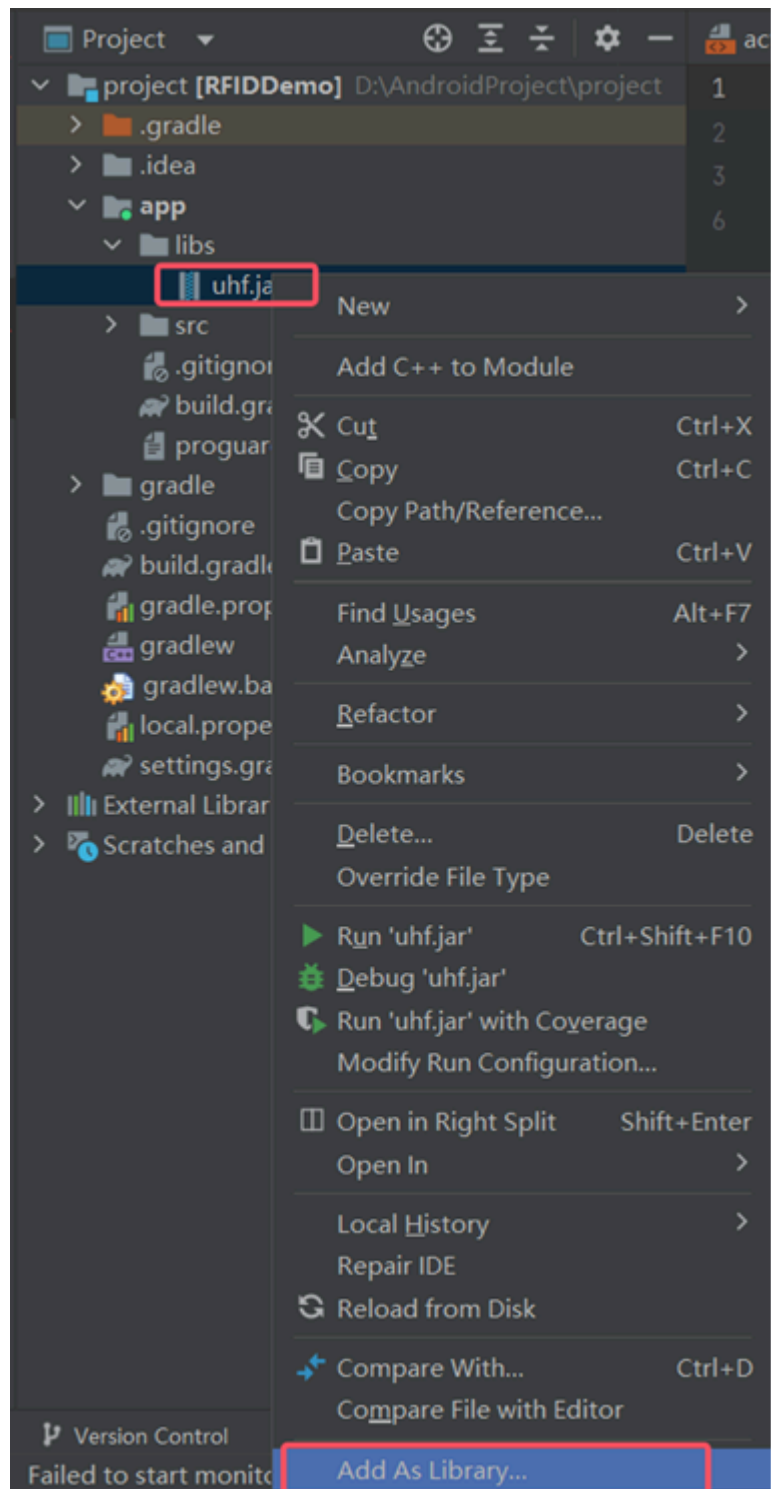
3. Set the package name, path, SDK version, etc., and click finish.

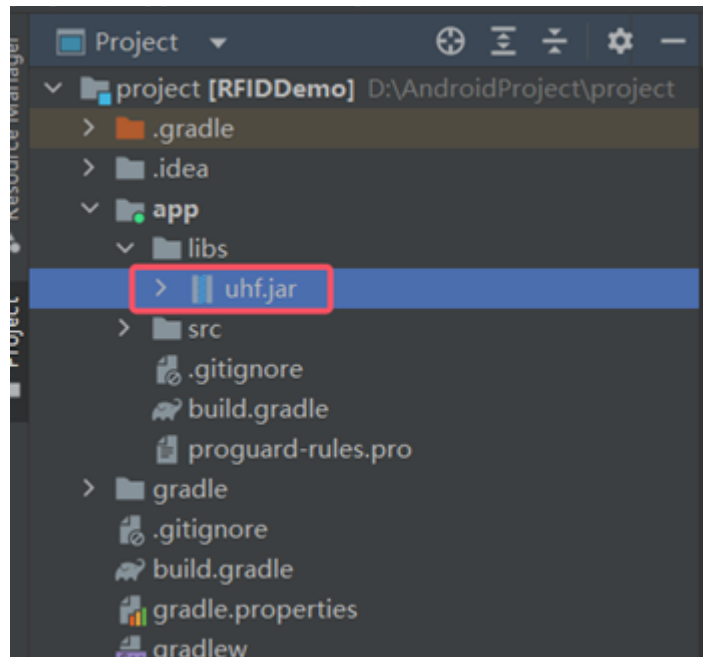


4. The SDK we provide is in the form of a jar package. Paste uhf.jar into project--app--libs.



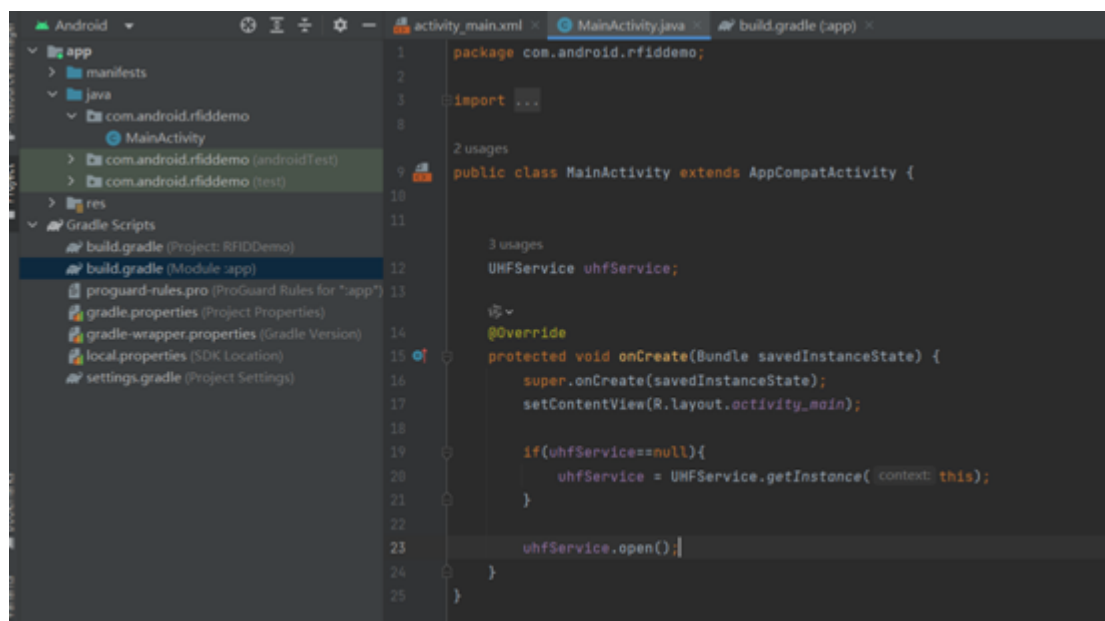
5. Right-click uhf.jar and select Add As Library.. to introduce the jar into the project.





```
implementation 'androidx.appcompat:appcompat:1.7.1'
implementation 'com.google.android.material:material:1.12.0'
implementation 'androidx.constraintlayout:constraintlayout:2.2.1'
implementation files('libs\\uhf.jar')
testImplementation 'junit:junit:4.13.2'
androidTestImplementation 'androidx.test.ext:junit:1.2.1'
androidTestImplementation 'androidx.test.espresso:espresso-core:3.6.1'
```

6. After the Jar package is imported, the relevant interfaces of RFID can be called normally.



Seuic RFID SDK for Android

Introduction

This article details how to use the Seuic RFID SDK to develop various basic and advanced features for Android applications.

The Seuic RFID Android SDK enables applications to communicate with RFID readers.

The Seuic RFID SDK provides APIs that allow external applications to manage connections to RFID readers and control connected RFID readers.

Basics

Note: Detailed API documentation can be found in the Java Class Reference Guide distributed with the Seuic RFID SDK.

The Seuic RFID SDK for Android provides the ability to manage RFID reader connections, perform various operations on connected RFID readers, configure connected RFID readers, and get other information related to connected RFID readers.

The Seuic RFID SDK for Android consists of a static Android library in "jar" format that should be linked with an external Android application. For instructions on configuring your Android application project to enable the Seuic RFID Android SDK, see "Importing the JAR Package" in the Connection Management section.

All available APIs are defined in the `com.seuic.uhf` package. The `UHFSERVICE` class in the `api` package is the root Java class in the SDK. Applications use a single instance of the `UHFSERVICE` object to interact with a specific reader.

Use the available readers and `UHFSERVICE` objects to register for events, connect to the reader, and, upon successful connection, perform the desired operations, such as taking inventory and accessing tags.

If an API call fails, an error message will be displayed. Detailed error information can be analyzed by capturing the logcat log.

Connection Management

Importing the JAR Package

Using Android Studio as an example:

In the application project, copy `uhf.jar` to the `libs` folder and reference it in `build.gradle`.

```
dependencies {  
    compileOnly files('libs/uhf.jar')  
}
```

Open Reader

Opening the reader is the first step to communicate with the RFID reader. Importing the package is the first step to using any API. Import the package as follows:

```
import com.seuic.uhf.UHFService;
```

Create an instance of the UHFService class and open the reader

```
mService = UHFService.getInstance(this);  
mService.open();
```

Close Reader

When the application completes the connection and operation with the RFID reader, it calls the following API to close the connection and release and clean up resources.

```
mService.close();
```

Config the reader

Set Power

Different reader chips support different power ranges. U6/U7/U8/U9 support a power range of 1~33dBm, and U4/U5 support a power range of 1~30dBm.

```
//set power to 30 dBm  
mService.setPower(30);
```

Get Power

```
mService.getPower();
```

Set Link Mode

Different reader chips support different link modes. Please refer to the table below for details.

```
//set link mode(0:P0, 1:P1, 2:P2, 3:P3 ... 25:P25)  
mService.setParameters(UHFService.PARAMETER_LINK_PROFILE, 1);
```

link mode table:

Profile	U8 (R2000)	U4/U6	U9 (E710 supports Gen2X)	U5(E510 supports Gen2X)	U7	applicable protocol
P0	40k FM0 Tari 25us	160k M8 Tari 12.5us	160k M8 Tari 20us/185	160k M8 Tari 20us/185	300k M8 Tari 25us	Gen2
P1	250k M4 Tari 25us	160k FM0 Tari 12.5us	640k M4 Tari 6.25us/148	640k M4 Tari 6.25us/148	300k M4 Tari 25us	Gen2
P2	300k M4 Tari 25us	250k FM0 Tari 12.5us	640k FM0 Tari 6.25us/103	/	640k FM0 Tari 6.25us	Gen2
P3	400k FM0 Tari 6.25us	320k M4 Tari 6.25us	250k M4 Tari 20us/146	250k M4 Tari 20us/146	250k M4 Tari 25us	Gen2
P4	/	/	640/M2/pie: 1.5/ tari_us: 7.5/120	640/M2/pie: 1.5/ tari_us: 7.5/120	/	Gen2
P5	/	/	160/M8/pie: 2.0/ tari_us: 20/382	160/M8/pie: 2.0/ tari_us: 20/382	/	Gen2
P6	/	/	250/M4/pie: 2.0/ tari_us: 20/343	250/M4/pie: 2.0/ tari_us: 20/343	/	Gen2
P7	/	/	640/M4/pie: 1.5/ tari_us: 7.5/345	640/M4/pie: 1.5/ tari_us: 7.5/345	/	Gen2
P8	/	/	640/M2/pie: 2.0/ tari_us: 7.5/323	640/M2/pie: 2.0/ tari_us: 7.5/323	/	Gen2
P9	/	/	640/M1/pie: 2.0/ tari_us: 7.5/302	/	/	Gen2
P10	/	/	160/M8/pie: 2.0/ tari_us: 20/4185	160/M8/pie: 2.0/ tari_us: 20/4185	/	Gen2X
P11	/	/	250/M4/pie: 2.0/ tari_us: 20/4146	250/M4/pie: 2.0/ tari_us: 20/4146	/	Gen2X
P12	/	/	640/M4/pie: 1.5/ tari_us: 7.5/4148	640/M4/pie: 1.5/ tari_us: 7.5/4148	/	Gen2X
P13	/	/	640/M2/pie: 2.0/ tari_us: 7.5/4124	640/M2/pie: 2.0/ tari_us: 7.5/4124	/	Gen2X
P14	/	/	[103, 148, 146, 185]/1120	/	/	Gen2
P15	/	/	[120, 148, 146, 185]/1122	[120, 148, 146, 185]/1122	/	Gen2
P16	/	/	[302, 345, 343, 382]/1320	/	/	Gen2
P17	/	/	[323, 345, 343, 382]/1322	[323, 345, 343, 382]/1322	/	Gen2
P18	/	/	[4185, 185]/5185	[4185, 185]/5185	/	Gen2+Gen2X
P19	/	/	[4146, 146]/5146	[4146, 146]/5146	/	Gen2+Gen2X
P20	/	/	[4148, 148]/5148	[4148, 148]/5148	/	Gen2+Gen2X
P21	/	/	[4124, 124]/5124	[4124, 124]/5124	/	Gen2+Gen2X
P22	/	/	[4382, 382]/5382	[4382, 382]/5382	/	Gen2+Gen2X
P23	/	/	[4343, 343]/5343	[4343, 343]/5343	/	Gen2+Gen2X
P24	/	/	[4345, 345]/5345	[4345, 345]/5345	/	Gen2+Gen2X
P25	/	/	[4323, 323]/5323	[4323, 323]/5323	/	Gen2+Gen2X
P26	/	/	/	/	/	Gen2
P27	/	/	Auto Profile	Auto Profile	/	Gen2X
P28	/	/	160/M8/pie: 2.0/ tari_us: 20/4185+CC1/2	160/M8/pie: 2.0/ tari_us: 20/4185+CC1/2	/	Gen2X
P29	/	/	250/M4/pie: 2.0/ tari_us: 20/4146+CC1/2	250/M4/pie: 2.0/ tari_us: 20/4146+CC1/2	/	Gen2X
P30	/	/	640/M4/pie: 1.5/ tari_us: 7.5/4148+CC1/2	640/M4/pie: 1.5/ tari_us: 7.5/4148+CC1/2	/	Gen2X
P31	/	/	640/M2/pie: 2.0/ tari_us: 7.5/4124+CC1/2	640/M2/pie: 2.0/ tari_us: 7.5/4124+CC1/2	/	Gen2X
P32	/	/	160/M8/pie: 2.0/ tari_us: 20/4382+CC1/2	160/M8/pie: 2.0/ tari_us: 20/4382+CC1/2	/	Gen2X
P33	/	/	250/M4/pie: 2.0/ tari_us: 20/4343+CC1/2	250/M4/pie: 2.0/ tari_us: 20/4343+CC1/2	/	Gen2X
P34	/	/	640/M4/pie: 1.5/ tari_us: 7.5/4345+CC1/2	640/M4/pie: 1.5/ tari_us: 7.5/4345+CC1/2	/	Gen2X
P35	/	/	640/M2/pie: 2.0/ tari_us: 7.5/4323+CC1/2	640/M2/pie: 2.0/ tari_us: 7.5/4323+CC1/2	/	Gen2X
P36	/	/	Auto Profile	Auto Profile	/	Gen2X

Note: The first column of this table lists all profile values. Columns two through six explain the specific profile values for different RFID chips. The last column describes the applicable protocol.

Note: To make the profile value setting take effect, you must call the setParameters function to enable the PAR AMETER_S2_ALGORITHM.

Set session and target

```
//session(0:S0, 1:S1, 2:S2, 3:S3)
mService.setParameters(UHFService.PARAMETER_INVENTORY_SESSION, 1);

//target(0:A, 1:B, 2:A/B)
mService.setParameters(UHFService.PARAMETER_INVENTORY_SESSION_TARGET,0)
```


Set frequency band

Different reader chips support different frequency band. Please refer to the table below for details.

```
mService.setRegion("FCC");
```

frequency band table:

Global regional RFID frequency band division and support status of different models								
region	frequency band MHz	U9(E710)		U5(E510)		U8(R2000)		U4/U6/U7
		US version(EU(Upper Band)/CH version	EU(Lower Band) version	US version(EU(Upper Band)/CH version	EU(Lower Band) version	US version(EU(Upper Band)/CH version	EU(Lower Band) version	CH version
China1	China1 (920~925)	support		support		support		support
FCC	FCC (902~928)	support		support		support		support
JAPAN	Japan (916.7~920.9)	support		support				
MALAYSIA	Malaysia (919~923)	support		support				
ETSI	ETSI(865~868)		support		support		support	
VIETNAM	Vietnam (920~923)	support		support				
AUSTRALIA	Australia (920~926)	support		support				
AUSTRIA	Austria (916.1~918.9)	support		support				
BRAZIL	Brazil (915~928)	support		support				
ESTONIA	Estonia (915~921)	support		support				
HK	HK (920~925)	support		support		support		support
INDIA	India (865~867)		support		support			
INDONESIA	Indonesia (923~925)	support		support				
ISRAEL	Israel (915~917)	support		support				
KOREA	Korea (917~920.8)	support		support				
NEW_ZEALAND	New Zealand (920~928)	support		support				
PERU	Peru (915~928)	support		support				
PHILIPPINES	Philippines (918~920)	support		support				
RUSSIA	Russia (915~928)	support		support				
SINGAPORE	Singapore (920~925)	support		support				
SOUTH_AFRICA	south africa (915.4~919)	support		support				
TAIWAN	Taiwan (922~928)	support		support				
THAILAND	Thailand (920~925)	support		support				
URUGUAY	uruguay (902~928)	support		support				

Get regional frequency band

```
mService.getRegion();
```

Set fixed frequency

The reader sets a fixed frequency, which is not saved upon power failure.
To reset the frequency to hopping, call the setRegion interface.

```
//set the frequency to 915.250 MHz
mService.setParameters(UHFService.PARAMETER_FREQUENCY, 915250);
```

Hide the PC value of the EPC area

Hide PC value: Inventory returns only the EPC of the tag

```
//hide the PC value
mService.setParameters(UHFService.PARAMETER_HIDE_PC, 1);

//do not hide the PC value
mService.setParameters(UHFService.PARAMETER_HIDE_PC, 0);
```

Other parameters

setParameters can set a variety of parameters. The specific parameters are defined in the following table:

id (Function parameter name)	U8(R2000)	U4/U6	U7	U9 (E710)	U5 (E510)	Remark
	value	value	value	value	value	
PARAMETER_LINK_PROFILE (0)	Refer to the sheet "Profile Definition"	Refer to the sheet "Profile Definition"	Refer to the sheet "Profile Definition"	Refer to the sheet "Profile Definition"	Refer to the sheet "Profile Definition"	Profile
PARAMETER_INVENTORY_SESSION (1)	0,1,2,3(S0,S1,S2,S3)	0,1,2,3(S0,S1,S2,S3)	0,1,2,3(S0,S1,S2,S3)	0,1,2,3(S0,S1,S2,S3)	0,1,2,3(S0,S1,S2,S3)	Session
PARAMETER_INVENTORY_SESSION_TARGET (2)	0,1(A,B)	0,1(A,B)	0,1,2(A,B,AB)	0,1,2(A,B,AB)	0,1,2(A,B,AB)	Target
PARAMETER_INVENTORY_SPEED (3)	0,1(fast,slow)	/	/	/	/	Inventory speed
PARAMETER_EXTENSIONS_FASTID (5)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	Fastid
PARAMETER_CLEAR_EPCLIST_WHEN_START_INVENTORY (6)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	Whether to clear the previous EPC before starting to search for the card
PARAMETER_HIDE_PC (7)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	Hide the PC value of the EPC area
PARAMETER_CLEAR_EPCLIST (8)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	Clear EPC list
PARAMETER_ALGORITHM_START_QVALUE (9)	0~15(4)	0~15(4)	0~15(4)	0~15(4)	0~15(4)	Initial Q value
PARAMETER_ALGORITHM_MINQ_VALUE (10)	0~15(0)	/	/	/	/	Minimum Q value
PARAMETER_ALGORITHM_MAXQ_VALUE (11)	0~15(15)	/	/	/	/	Maximum Q value
PARAMETER_ALGORITHM_TOGGLE_TARGET (13)	0,1(DISABLE,ENABLE)	/	/	/	/	Toggle target
PARAMETER_EXTENSIONS_TAGFOCUS (15)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	TagFocus
PARAMETER_S2_ALGORITHM(34)	/	/	/	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	If enabled, the target, session, and profile parameters are configurable and will be set to the desired value. If not enabled, the target, session, and profile parameters are adaptive to the scenario and the settings will not take effect.
PARAMETER_TAG_EMBEDDEDATA (30)	EmbeddedData	EmbeddedData	EmbeddedData	EmbeddedData	EmbeddedData	additional data, such as Reserved, TID or User
PARAMETER_TAG_FILTER (31)	TagFilter	TagFilter	TagFilter	TagFilter	TagFilter	Set filtering conditions

Note: The first column of this table lists all the parameters of the setParameters function. The second to sixth columns explain the values that can be set for different RFID chips. The last column explains the specific meaning of each parameter.

Basic Operations

Tag operations require importing the EPC class

```
import com.seuic.uhf.EPC;
```

The tag information includes the following:

```
public class EPC {
```

```

    public byte[] id;
    public int len;
    public int rssi;
    public int count;
    //get epcID
    public String getId() {
        throw new RuntimeException("Stub!");
    }
    //get additional data
    public String getEmbedded() {
        throw new RuntimeException("Stub!");
    }
}

```

Single Inventory Tag

A single inventory only returns one tag data

```

EPC epc = new EPC();
mService.inventoryOnce(epc, 0);

```

Callback Function

Continuous inventory uses callback functions to continuously return tag data.

Register the callback function before starting the inventory and unregister it when the program ends.

When receiving callback data, avoid performing time-consuming operations in the tagsRead function.

```

//callback function
private IReadTagsListener mIReadTagsListener = new IReadTagsListener() {
    @Override
    public void tagsRead(List<EPC> epclist) {
        //data returns to epclist
    }
};

//register callback
mService.registerReadTags(mIReadTagsListener);

//unregister callback
mService.unregisterReadTags(mIReadTagsListener);

```

Start Continuously Inventory

```

mService.inventoryStart();

```

Stop Continuously Inventory

```

mService.inventoryStop();

```

Read Tags

The application calls the `mService.readTagData` function to read data from the memory of the specified EPC tag.

```
String epcID = "1234ABCD000000000000001122";//epc
String psw = "00000000";//password
int bank = 3;//0:Reserved, 1:EPC, 2:TID, 3:User
int offset = 0;//Read from byte 0
int len = 2;//Read 2 bytes of data
byte[] data = new byte[2];//Defined for saving written data
mService.readTagData(getHexByteArray(epcID),
                    getHexByteArray(psw),
                    bank,
                    offset,
                    len,
                    data);
```

Convert a string into a hexadecimal array

```
public static byte[] getHexByteArray(String hexString) {
    if (hexString == null || hexString.equals("")) {
        return null;
    }
    hexString = hexString.toUpperCase();
    hexString = hexString.replace(" ", "");
    byte[] buffer = new byte[hexString.length() / 2];
    int length = hexString.length() / 2;
    char[] hexChars = hexString.toCharArray();
    for (int i = 0; i < length; i++) {
        int pos = i * 2;
        buffer[i] = (byte) (charToByte(hexChars[pos]) << 4 | charToByte(hexChars[pos + 1]));
    }
    return buffer;
}
```

Write Tags

The application calls the `mService.writeTagData` function to write data to the specified EPC tag memory.

```
String epcID = "1234ABCD000000000000001122";//epc
String psw = "00000000";//password
int bank = 3;//0:Reserved, 1:EPC, 2:TID, 3:User
int offset = 0;//Start writing from byte 0
int len = 2;//write 2 bytes of data
byte[] data = new byte[2];//Defined for saving written data
mService.readTagData(getHexByteArray(epcID),
                    getHexByteArray(psw),
                    bank,
                    offset,
                    len,
                    data);
```

Lock Tags

The application calls the mService.lockTag function to lock the memory of the specified EPC tag.

```
String epcID = "1234ABCD000000000000001122";//epc
String psw = "00000000";//password
//0,1,2,3 unlock tag (0: Reserved 1: EPC 2: TID 3: User)
//10,11,12,13 lock tag (10: Reserved 11: EPC 12: TID 13: User)
//20,21,22,23 lock tag permanently (20: Reserved 21: EPC 22: TID 23: User)
//30,31,32,33 unlock tag permanently (30: Reserved 31: EPC 32: TID 33: User)
int mLockFlag = 13;
mService.lockTag(getHexByteArray(epcID),
                getHexByteArray(psw),
                mLockFlag);
```

Kill Tags

The application calls the mService.killTag function to kill a tag. A killed tag cannot be restored.

```
String epcID = "1234ABCD000000000000001122";//epc
String psw = "12345678";//The destruction password must be set to a non-zero password first
mService.killTag(getHexByteArray(epcID),
                getHexByteArray(psw));
```

Advanced Operations

Filter Tags

Set filter conditions so that only tag data that meets the conditions is returned

```

byte[] val = new byte[255];
val[0] = 1;//1:EPC, 2:TID, 3:User
val[1] = 0;//Matching starts at byte 0
val[2] = 4;//Matching 4 bytes of data
val[3] = 0;//0:match, 1: mismatch
String filterdata = "11223344";
byte[] data = getHexByteArray(filterdata);
System.arraycopy(data, 0, val, 4, val[2]);
mService.setParamBytes(UHFService.PARAMETER_TAG_FILTER, val);

```

Adding additional data when inventory tags

When taking inventory of tags, the tag EPC data will be obtained. After enabling this function, data in other storage areas of the tag, such as Reserved, TID, or User, can be read at the same time. Only data in one storage area can be read at a time.

```

byte[] embd = new byte[255];
embd[0] = 2;//0:Reserved, 2:TID, 3:User
embd[1] = 0;//Starting from byte 0
embd[2] = 12;//Append 12 bytes of data
String psw = "00000000";//password
System.arraycopy(getHexByteArray(psw), 0, embd, 3, 4);
mService.setParamBytes(UHFService.PARAMETER_TAG_EMBEDDEDATA, embd);

```

Temperature tags and LED tags

Temperature tags

The algorithm for each temperature tag is different, so before reading a temperature tag, you need to specify the tag model. The currently implemented temperature tag models are as follows:

model	TID	values
B5020M1-T1E	E201E2	1
LTU32	E2035101	2
NMV2D	E2C19CB1	3
ET-C2ES	E201E2	4
ET-C2ESL	FFFFFF	5
M3D	E28240	6
HX03	E200FF	8

```
String epcID = "1234ABCD000000000000001122";//epc
String psw = "00000000";//password
int Manufacturer = 1;//The parameter values ••corresponding to model B5020M1-T1E in the
above figure
mService.readTagTemperature(getHexByteArray(epcID),
                           getHexByteArray(psw),
                           Manufacturer);
```

LED tags

Each LED tag has a different algorithm. Therefore, before reading a temperature tag, you need to identify the tag model. Currently implemented temperature tag models are as follows:

model	TID	method to light up the tags
KX2005X-BL	E281D0	Reserved area, offset 8 bytes, read 2 bytes
N2ESL	E201E2	User area, offset 224 bytes, read 2 bytes
VBL	E2C19C	Reserved area, offset 10 bytes, read 2 bytes

An example of lighting up the KX2005X-BL model label is as follows:

```
//set the additional data
byte[] embd = new byte[255];
embd[0] = 0;
embd[1] = 8;
embd[2] = 2;
String psw = "00000000";
System.arraycopy(getHexByteArray(psw), 0, embd, 3, 4);
mService.setParamBytes(UHFService.PARAMETER_TAG_EMBEDDEDATA, embd);
//start inventory
mService.inventoryStart();
```

LED tags always on (only supported by U9 chipset)

The LED tag remains on continuously, without flickering.

Maintaining a constant on state consumes more power and increases the temperature of the RFID module. It is recommended not to leave the tag on for extended periods.

The following example shows how to achieve a constant on state for the KX2005X-BL model tag:

```
//Each time the module is powered on, it is necessary to set the constant light mode
byte[] startCMD = new byte[1];
byte[] resCMD = new byte[1];
startCMD[0] = 0x08;
boolean ret = mService.IOControl(47,startCMD,1,resCMD,1,100);
//Set the fixed frequency to 915.250MHz
mService.setParameters(UHFService.PARAMETER_FREQUENCY, 915250);
//Setting additional data
byte[] embd = new byte[255];
```

```

embd[0] = 0;
embd[1] = 8;
embd[2] = 2;
String psw = "00000000";
System.arraycopy(getHexByteArray(psw), 0, embd, 3, 4);
mService.setParamBytes(UHFSERVICE.PARAMETER_TAG_EMBEDDEDATA, embd);
//start inventory
mService.inventoryStart();

```

Extended interface

read tags

The application calls the `mService.readTagDataEx` function to specify the EPC, TID, or User area to read data.

```

//Specify filtering TID
String TID = "E280110C20007582C3480A11";//TID
int filterbank = 2;//1:EPC, 2:TID, 3:User
int filteroffset = 0;//Matching starts at byte 0
int filterlen = 12;//Matches 12 bytes of data
//Read tag parameters
String psw = "00000000";//password
int bank = 3;//0:Reserved, 1:EPC, 2:TID, 3:User
int offset = 0;//Read from byte 0
int len = 2;//Read 2 bytes of data
byte[] data = new byte[2];//Definition for saving read data
mService.readTagData(getHexByteArray(TID),
                    filterbank,
                    filteroffset,
                    filterlen,
                    getHexByteArray(psw),
                    bank,
                    offset,
                    len,
                    data);

```

write tags

The application calls the `mService.writeTagDataEx` function to specify the EPC, TID, or User area to write data.

```

//Specify filtering TID
String TID = "E280110C20007582C3480A11";//TID
int filterbank = 2;//1:EPC, 2:TID, 3:User
int filteroffset = 0;//Matching starts at byte 0
int filterlen = 12;//Matches 12 bytes of data
//Write tag parameters
String psw = "00000000";//password
int bank = 3;//0:Reserved, 1:EPC, 2:TID, 3:User
int offset = 0;//Start writing from byte 0
int len = 2;//write 2 bytes of data

```



```
byte[] data = new byte[2]; //Used to save written data
mService.readTagData(getHexByteArray(TID),
                    filterbank,
                    filteroffset,
                    filterlen,
                    getHexByteArray(psw),
                    bank,
                    offset,
                    len,
                    data);
```

lock tags

The application calls the `mService.lockTagEx` function to specify the EPC, TID, or User area to lock the tag.

```
//Specify filtering TID
String TID = "E280110C20007582C3480A11"; //TID
int filterbank = 2; //1:EPC, 2:TID, 3:User
int filteroffset = 0; //Matching starts at byte 0
int filterlen = 12; //Matches 12 bytes of data
//Lock tag parameters
String psw = "00000000"; //password
//0,1,2,3 Unlock Tags (0: Reserved 1: EPC 2: TID 3: User)
//10,11,12,13 Lock Tag (10: Reserved 11: EPC 12: TID 13: User)
//20,21,22,23 Permanent lock tags (20: Reserved 21: EPC 22: TID 23: User)
//30,31,32,33 Permanently unlock tags (30: Reserved 31: EPC 32: TID 33: User)
int mLockFlag = 13;
mService.lockTag(getHexByteArray(TID),
                filterbank,
                filteroffset,
                filterlen,
                getHexByteArray(psw),
                mLockFlag);
```

kill tags

An application can call `mService.killTagEx` to specify the EPC, TID, or User area to destroy a tag. A destroyed tag cannot be restored.

```
//Specify filtering TID
String TID = "E280110C20007582C3480A11"; //TID
int filterbank = 2; //1:EPC, 2:TID, 3:User
int filteroffset = 0; //Matching starts at byte 0
int filterlen = 12; //Matches 12 bytes of data
//Kill tag parameters
String psw = "00000000"; //To destroy the password, you must first set a non-zero password
mService.killTag(getHexByteArray(TID),
                filterbank,
                filteroffset,
                filterlen,
                getHexByteArray(psw));
```

Gen2X features (only supported by U5&U9)

TagFocus

TagFocus can significantly improve reading and writing efficiency and accuracy, especially in high-density tag environments.

This feature enables the reader to instruct the tag to continuously refresh its A/B flag status, thereby maintaining a non-responsive state. By commanding the inventoried tags to remain silent while searching for uninventored tags, the reader's success rate in inventorying difficult-to-read tags will be significantly improved.

```
//0:close, 1: open  
mService.setParameters(UHFService.PARAMETER_EXTENSIONS_TAGFOCUS, 1);
```

FastID

The FastID feature requires that all tag chips append their TID to the EPC during the inventory process.

```
//0:close, 1: open  
mService.setParameters(UHFService.PARAMETER_EXTENSIONS_FASTID, 1);
```

Close range mode

Close range mode reduces the read range by 90% to prevent tag data from being read at a distance.

Here's how to do it:

Set a non-zero password first.

Use a non-zero password to read the password area, the address offset is 8 bytes, read 2 bytes, and modify bit[4]=1 of the read data.

```
c068: 1100 0000 0110 1000  
c078: 1100 0000 0111 1000
```

Write the modified data back to the password area with a non-zero password, the address offset is 8 bytes, write 2 bytes

If you need to disable this function, modify bit[4]=0.

```
//Turn on close range mode
//Read out the password area first offset: 8 len: 2
String epcID = "1234ABCD000000000000001122";//epc
String psw = "12345678";//password
byte[] data = new byte[2];
mService.readTagData(getHexByteArray(epcID), getHexByteArray(psw), 0, 8, 2, data);
data[1] = (byte)(data[1] | (1 << 4));
mService.writeTagData(getHexByteArray(epcID), getHexByteArray(psw), 0, 8, 2, data);
//Turn off close range mode
data[1] = (byte)(data[1] & ~(1 << 4));
mService.writeTagData(getHexByteArray(epcID), getHexByteArray(psw), 0, 8, 2, data);
```

Protected Mode

Impinj Protected Mode makes the tag invisible to the reader and visible to the reader using a security PIN

Here's how to do it:

Set a non-zero password first.

Use a non-zero password to read the password area, address offset 8 bytes, write 2 bytes, and modify bit[1]=1.

```
C068: 1100 0000 0110 1000
C06A: 1100 0000 0110 1010
```

Write the modified data back to the password area with a non-zero password, the address offset is 8 bytes, and 2 bytes are written.

After the protection mode is turned on, the normal inventory cannot find the tag, and the filtered access password inventory is required. The filtered access password parameters are as follows: target=0, action=1, bank=3, offset=0, len=4, data=4-byte password.

If you need to disable this function, modify bit[1]=0.

```
//Enable Protected Mode
//Read out the password area first offset: 8 len: 2
String epcID = "1234ABCD000000000000001122";//epc
String psw = "12345678";//password
byte[] data = new byte[2];
mService.readTagData(getHexByteArray(epcID), getHexByteArray(psw), 0, 8, 2, data);
data[1] = (byte)(data[1] | (1 << 1));
mService.writeTagData(getHexByteArray(epcID), getHexByteArray(psw), 0, 8, 2, data);
//Disable Protected Mode
//Read out the password area first offset: 8 len: 2
data[1] = (byte)(data[1] & ~(1 << 1));
mService.writeTagDataEx(3,0,4,getHexByteArray(psw), getHexByteArray(psw), 0, 8, 2, data);
//After enabling protection mode, you must filter the password area before inventorying the tags
byte[] val = new byte[255];
val[0] = 0;//target
val[1] = 1;//action
```

```
val[2] = 3;//bank user
val[3] = 0;//offset
val[4] = 4;//Password length
val[5] = 0;//match
byte[] data = getHexByteArray(psw);
System.arraycopy(data, 0, val, 6, val[4]);
mService.setParamBytes(27, val);
```

Tag Authentication

Obtain the encrypted data of the tag and verify the authenticity of the tag.

Here's how to do it:

Send a 6-byte random number to the tag, and the tag returns a 12-byte TID + 8-byte encrypted data

The first 6 bits of the first byte of the 6-byte random number must be 00001, indicating that 64-bit TID data is returned.

Return data format: 1 byte TID length + TID data + 1 byte encrypted data length + encrypted data

```
byte[] sendCMD = new byte[11];
byte[] resCMD = new byte[32];
String random = "051122334455";
sendCMD[0] = 1;//tag model 1:M775, 2:C899
sendCMD[1] = 1;//send_rep
sendCMD[2] = 1;//inc_rep_len
sendCMD[3] = 1;//csi
sendCMD[4] = 6;//data length
System.arraycopy(getHexByteArray(random), 0, sendCMD, 5, 6);//6-byte random number
mService.IOControl(52, sendCMD, 11, resCMD, 32, 0);
byte[] tid = new byte[12];
byte[] encryptedData = new byte[8];
System.arraycopy(resCMD, 1, tid, 0, resCMD[0]);
System.arraycopy(resCMD, resCMD[0]+1+1, encryptedData, 0, resCMD[resCMD[0]+1]);
```